YANTRIKA Robotics & Coding Club, IACS Kolkata

Contributing to the Games Arcade

A complete guide to improving our open-source games and adding your own, from first clone to merged pull request.

LEARN | BUILD | COMPETE | SHARE

Who is this for? Students, alumni, hobbyists and first-timers. You do not need to be a Yantrika member, and you do not need advanced tools: if you can open a web page and edit a text file, you can make a game.

Licence: every game is open source under the MIT licence. By contributing you agree your work is released under the same terms.

Repository: `github.com/yantrika-iacs/Yantrika-IACS.github.io`
Contact: contact@yantrika.in

Contents

1. The big picture 2
2. How the project is organised 2
3. Getting set up 3
4. Add a new game: the quick path 4
5. Anatomy of a game 5
6. Reference: YK.register fields 6
7. Rules every game follows 7
8. Thumbnails and assets 8
9. Improving an existing game 8
10. Checking your work and submitting 9
11. Licence, credit and third-party material 10
12. For reviewers and maintainers 10
13. Troubleshooting 11
14. Cheat sheet 11

1. The big picture

The Games Arcade on the Yantrika website helps visitors learn robotics, electronics, coding and game theory by playing. Every game is tied to one of our four pillars and to the events it prepares people for. The whole arcade is **free and open source**, and it is built so that adding or improving a game is a small, safe, self-contained change.

What you can do

- **Add a new game.** One folder, one JavaScript file, one picture, and one line in a list.
- **Improve an existing game:** fix a bug, add a harder level, sharpen the theory text, make it friendlier on phones, add keyboard or screen-reader support, or translate it.
- **Report a bug or suggest a game** through a GitHub issue.
- **Review other people's pull requests.** A second pair of eyes helps everyone learn.

Design principles

Principle	What it means for you
No build step	Plain HTML, CSS and JavaScript. No framework, bundler or package install is needed to make a game.
One folder per game	Everything for your game lives in `games/<id>/`. Your changes rarely collide with anyone else's.
Works offline, no tracking	No network requests, cookies, analytics, accounts or leaderboards. Visitors' privacy is protected by design.
Safe to fail	If one game has an error, the rest of the arcade keeps working and the player sees a friendly message.
Checked automatically	A validator, a linter and a headless test run on every pull request, so you get fast feedback.

2. How the project is organised

The arcade lives in the `games/` folder of the website repository.

```
games/
  manifest.js        the list of games shown on the site, and their order
  core.js            small shared toolkit (esc, SFX, SL, TIN, YK.register ...)
  games.css          shared look of the arcade (cards, slider, full screen)
  CREDITS.md         third-party material, with licences
  README.md          developer guide (short version of this document)
  shared/            helper scripts several games can use (figures.js)
  _template/         a working example. Copy this to start
  circuit/           one folder per game, for example:
    game.js            the game itself
    thumb.png          its 800x800 picture
  binary/ walk/ echo/ bot/ breadboard/ pd/ duel/ simon/ maze/ rps/ auction/
  pick/ plan/
tools/
  new-game.js        scaffolds a new game for you
  check-games.js     validates every game (no install needed)
  smoke-test.js      opens every game in a headless browser
.github/             issue forms, pull request template, automatic checks
CONTRIBUTING.md      the rules in short form
docs/GAME_IDEAS.md   ideas if you do not know what to build
```

How a game reaches the screen

1. The page loads `games/core.js` and `games/manifest.js`.
2. The loader fetches every `games/<id>/game.js` listed in the manifest, in order.
3. Each game file calls `YK.register({...})` to describe itself and hand over its `run` function.
4. The home page slider, the Games page, and the related event pages are all built from that registry, so your game appears everywhere automatically.
5. When a visitor opens your game, the site calls `run(el)` with an empty box. You draw the game into it.

3. Getting set up

You need three things: a GitHub account, Git (or just the GitHub website), and a text editor such as VS Code. **Node.js 18 or newer** is needed only for the helper tools; the website itself needs nothing.

Step by step

1. **Fork** the repository on GitHub (the Fork button, top right). This makes your own copy.
2. **Clone** your fork to your computer.
3. **Open the site** in a browser, either by double-clicking `index.html` or by serving the folder (recommended).

```
git clone https://github.com/<your-username>/Yantrika-IACS.github.io
cd Yantrika-IACS.github.io

# serve the site locally, then visit http://localhost:8000
python3 -m http.server 8000
```

Tip: no Git? No problem.
Open any `games/<id>/game.js` on GitHub, press the pencil icon, edit, and GitHub will create the fork and the pull request for you. This is perfect for typo fixes and small improvements.

Refreshing while you work

Browsers cache files. After each edit press **Ctrl+F5** (Cmd+Shift+R on a Mac) to reload everything. If a game seems unchanged, this is almost always the reason.

4. Add a new game: the quick path

The scaffolding tool does the boring parts for you.

```
node tools/new-game.js sort-race "Sorting Race" compete
#                      ^id        ^title        ^pillar
# pillar is one of: learn  build  compete  share
```

This creates `games/sort-race/` from the template and adds `sort-race` to `games/manifest.js`. Then:

1. Open the site, go to **Games**, and play your new game. It is a small working example (Ohm's law quiz).
2. Edit `games/sort-race/game.js`. Save, press Ctrl+F5, play again. Repeat.
3. Replace the placeholder `thumb.svg` with your own picture (see section 8).
4. Update the metadata: title, one-line description, the events it prepares people for, and your name in `authors`.
5. Run `node tools/check-games.js` until it reports 0 errors.
6. Commit on a branch and open a pull request (section 10).

Choosing the id

The **id** is your folder name and your game's identity. Use lowercase letters, numbers and dashes only (`sort-race`, `pwm-lab`). It must be unique and must match the folder name exactly. The names `template` and `shared` are reserved.

Choosing the pillar

Pillar	Use it for
learn	Teaching a concept: circuits, binary, sensors, algorithms.
build	Assembling or wiring something, designing a robot or system.
compete	Games with a score, a rival bot or a clock: hackathon practice.
share	Teaching others, outreach, planning workshops and mentoring.

Not sure what to build?
See `docs/GAME_IDEAS.md` for starters grouped by pillar: resistor colour codes, a PID line-follower tuner, a sorting race, a pinout puzzle, and more. Open a **Game idea** issue first so two people do not build the same thing.

5. Anatomy of a game

Here is the smallest useful game. Everything is wrapped in a function so its names never clash with another game's.

```
(function(){
'use strict';

function play(el){               // el = an empty box on the page
  let taps=0;
  el.innerHTML=`
    <h2 class="m">Tap counter</h2>
    <button class="btn cy" id="go" type="button">Tap me</button>
    <p id="out" aria-live="polite">0 taps</p>`;
  el.querySelector('#go').onclick=()=>{
    taps++;
    SFX.play('coin');
    el.querySelector('#out').textContent=taps+' taps';
  };
}

YK.register({
  id:'tap-counter',              // = folder name
  pillar:'compete',
  icon:'\ud83d\udc46',          // one emoji
  title:'Tap Counter',
  description:'Tap as many times as you can.',
  events:['robotics-hackathon'], // ids from EV in index.html, or []
  thumb:'games/tap-counter/thumb.png',
  run:play
});
})();
```

The three ingredients

- **The wrapper** `(function(){ ... })();` keeps your variables private.
- **The game function** receives `el` and draws into it. It is called every time the game is opened, so set up fresh state inside it. State kept outside the function (for example a "best score") survives until the page is reloaded.
- **The registration** `YK.register({...})` describes the game to the site.

Adding theory and a "Try it" demo

Every game can teach as well as entertain. Add `theory` (HTML text shown under the game) and optionally `demo`, a small interactive experiment:

```
theory:`<p><b>Ohm's law</b> links voltage, current and resistance: I = V / R.</p>
        <ul><li>More resistance means less current.</li></ul>`,

demo:box=>TIN(box,'Try it: change the resistor',
  SL('r','Resistor (ohms)',50,1000,220,10),       // id, label, min, max, start, step
  v=>`Current at 5 V = <b>${(5/v('r')*1000).toFixed(1)} mA</b>`),
```

`SL(...)` makes a slider. `TIN(...)` builds the box and calls your function every time a slider moves; `v('r')` reads the slider whose id is `r`. See `games/circuit/game.js` and `games/echo/game.js` for more examples.

Game-specific CSS

Put styles for your game in the `css` field as a string. Prefix every class with your game id so nothing clashes:

```
css:`.tapcounter-big{font-size:3rem;text-align:center}
     .tapcounter-row{display:flex;gap:10px;flex-wrap:wrap}`,
```

6. Reference: YK.register fields

Field	Required	Meaning
`id`	yes	Lowercase letters, numbers, dashes. Must equal the folder name.
`pillar`	yes	`learn`, `build`, `compete` or `share`.
`title`	yes	Game name. Keep it under 40 characters.
`description`	yes	One line for the game card, ideally under 110 characters.
`run(el)`	yes	Function that draws the game into `el`. Called each time the game opens.
`icon`	no	One emoji. Defaults to a game controller.
`events`	no	List of event ids the game prepares people for. The game is then listed on those event pages. Valid ids are in the EV list in `index.html`.
`thumb`	no	Path to your picture inside your folder. Defaults to `games/<id>/thumb.png`.
`theory`	no	HTML shown under the game as "The theory behind it".
`demo(box)`	no	Small interactive "Try it" shown inside the theory box.
`css`	no	CSS text for your game only. Prefix class names.
`authors`	no	List of names or GitHub handles, for example `['@octocat']`. Shown on the game's page.

Tools available inside every game

Name	What it does
`esc(text)`	Makes text safe to put into `innerHTML`. Use it on anything that is not your own fixed text.
`SFX.play(name)`	Sound effects: click, on, off, step, bump, hint, tick, ping, go, tie, coin, win, lose, buzz, gavel, whoosh, devil. The player's mute button is respected automatically.
`SL(id,label,min,max,value, step)`	Slider for a "Try it" demo.
`TIN(box,title,html,fn)`	The "Try it" box. `fn(v)` returns HTML; `v('id')` reads a slider or select.
`YK.figure(x,hat,mood)`	Cartoon character as SVG. Moods: normal, happy, sad, angry, devil.
`EV, LINKS, PIL`	Site data: events, links, pillars. Read only.

Look and feel

- **Colours:** use the site's CSS variables so dark mode works: `--bg --card --ink --dk --soft --red --yel --cy --bl --or --pk`.
- **Ready-made classes:** `btn`, `btn cy`, `chip` (toggle buttons; set `aria-pressed`), `info` (note box), `bar` (progress bar), `m` (marker-style heading).
- **Responsive:** games open full screen on phones, tablets and laptops. Use wrapping flex rows, `max-width:100%`, and tap targets of at least 40 px. Test a narrow window.
- **Accessible:** use real `<button>` elements, announce results with `aria-live="polite"`, never rely on colour alone, and respect `prefers-reduced-motion`.

7. Rules every game follows

The checker enforces most of these automatically, but they are worth understanding.

#	Rule	Why
1	**Works offline.** No `fetch`, XMLHttpRequest, WebSocket, external scripts, fonts or images.	Games must work in a classroom with poor internet and must not leak visitor data.
2	**No tracking or accounts.** No cookies, analytics, sign-ins or leaderboards.	The club removed score saving. Keep any "best score" in memory only.
3	**No personal data.** Browser storage, if used, holds only tiny local settings.	Respect for visitors, many of whom are school students.
4	**Clean up after yourself.** Stop timers and intervals when the game is closed (check `el.isConnected`).	Closing a game clears its box; stray timers would keep running.
5	**One folder.** Everything lives in `games/<id>/`. The only outside edit is one line in `manifest.js`.	Small, easy-to-review pull requests.
6	**Educational and kind.** Teach or celebrate robotics, electronics, computing, maths or game theory. Suitable for school students.	This is a learning space.
7	**Your work, or credited.** Only submit what you wrote or may share under MIT.	Keeps the project legally clean.
8	**Keep it small.** `game.js` under 150 KB, thumbnail under 500 KB.	Fast loading on phones.

8. Thumbnails and assets

- **Size and format:** about 800 x 800 pixels (square), PNG, JPG, WebP, AVIF or SVG, under 500 KB. SVG is great for simple artwork and stays crisp.
- **Location:** inside your game folder, for example `games/sort-race/thumb.png`, and referenced by the `thumb` field.
- **Style:** bold, simple and readable when small. The card shows your thumbnail with the game title over the bottom edge, so keep important details away from the bottom.
- **Other files** (sounds, extra images) also go in your folder. Keep each file under 2 MB and credit anything you did not make (section 11).
- **Never** link to images, fonts or scripts on other websites. The checker will reject it.

> **Making a picture without design tools**
> A hand-written SVG is a few lines of code and works well. The template ships with a placeholder `thumb.svg` you can edit in any text editor. Screenshots of your finished game also make good thumbnails.

9. Improving an existing game

You do not have to write a whole game. Every open game shows **Read the code**, **Edit it on GitHub** and **Report a bug** links under it. Good first contributions:

Contribution	Where to look
Fix a typo or unclear sentence in the theory text	`theory:` field in the game's `game.js`
Add a harder level or new opponent	The game function; follow the existing data structures (levels, opponent lists)
Improve phone layout	Layout rules in `games/games.css` or the game's own `css`
Keyboard or screen-reader support	Add key handlers, `aria-label`, `aria-live`, focus handling
Translate a game into Bengali or Hindi	Text strings inside the game; discuss the approach in an issue first
Add a "Try it" demo to a game that lacks one	`demo:` field; copy the pattern from `games/circuit`
Fix a bug you found while playing	Reproduce it, find the line, change it, test, open a pull request

Tips for editing someone else's game

- Keep the change **small and focused**: one fix or feature per pull request.
- Match the existing code style. Do not reformat the whole file.
- Play the game **before and after** your change to make sure nothing else broke.
- If you change how a game works, update its `theory` text if it describes the old behaviour.

10. Checking your work and submitting

Before you open a pull request

```
node tools/check-games.js      # fast, no install needed. Aim for 0 errors

npm install                    # once, to get the lint and test tools
npm test                       # validator + lint + opens every game headlessly
```

Check	What it catches
`check-games.js`	Folder not in manifest, id mismatch, bad pillar, missing title/description, unknown event id, missing or oversized thumbnail, banned patterns (network calls, `eval`, cookies, external files).
`npm run lint`	Typos and undefined variables, for example calling a function that does not exist.
`npm run smoke`	Opens every game in a headless browser and fails if any game throws an error.

Submitting a pull request

1. **Create a branch:** `git checkout -b game/sort-race`
2. **Commit:** `git add games` then `git commit -m "Add Sorting Race"`. Use a clear message that says what and why.
3. **Push:** `git push origin game/sort-race`
4. **Open the pull request** on GitHub from your branch to `main`. Fill in the template: what changed, which games, the checklist, and a screenshot if you can.
5. **Wait for the automatic checks.** A green tick means they passed. A red cross shows the log; fix the problem, commit again and push. The pull request updates itself.
6. **Respond to review.** A maintainer usually replies within a week and may ask for small changes. That is normal and friendly, not a rejection.
7. **Celebrate.** Once merged, your game goes live with the next site update and you are credited in Git history and, if you set `authors`, on the game's page.

Pull request checklist

- `node tools/check-games.js` passes
- I played start to finish, including on a narrow (phone-sized) window
- Works offline: no network requests, cookies, analytics or external files
- Everything is my own work, or credited in `games/CREDITS.md`
- I am happy for my work to be released under the MIT licence

11. Licence, credit and third-party material

- The project is released under the **MIT License** (file `LICENSE`). Anyone may use, copy, modify and share it, including commercially, as long as the licence notice is kept.
- By opening a pull request you confirm that **you wrote your contribution (or have the right to share it)** and that it is licensed under MIT. This covers code, text and pictures in your game folder.
- **You keep your copyright.** You are credited through Git history and optionally the `authors` field.
- Please do not use Yantrika or IACS logos in a way that suggests official endorsement of a fork.

Using someone else's material

An image, sound, snippet of code or algorithm description is allowed only if its licence is compatible with MIT, for example **CC0, CC BY, MIT, BSD or Apache-2.0**. When you use such material:

1. Keep any required notice or attribution.
2. Add a row to `games/CREDITS.md` with the game, the material, the author or source, and the licence.
3. Mention it in your pull request description.

> **Do not submit**
> Anything copied from a book, a website or another game whose terms do not allow reuse; screenshots of copyrighted games; or content from an AI tool whose terms do not let you release the output under MIT. When in doubt, ask in the pull request or leave it out.

12. For reviewers and maintainers

Every pull request runs the validator, the linter and the headless smoke test (`.github/workflows/check.yml`). Automation cannot judge everything, so reviewers should also check:

Area	Questions to ask
Play-through	Does it work start to finish? Can the player get stuck? Does it restart cleanly?
Phones	Does it fit a narrow screen? Are buttons easy to tap? Does it work in dark mode?
Accuracy	Is the theory text correct and at the right level for school students?
Privacy and safety	No network access, tracking or personal data. Content is suitable for young people.
Licensing	Is third-party material listed in `CREDITS.md` with a compatible licence?
Scope	Is the change focused? Does it touch only its own folder plus one manifest line?
Tone	Is the feedback kind and specific? First-time contributors deserve extra encouragement.

Merging

- Prefer **squash and merge** for a tidy history, keeping the contributor as the author.
- Thank the contributor by name. Invite them to review someone else's pull request next.
- Keep `games/manifest.js` ordered by pillar (learn, build, compete, share) when merging new games.

Community

Everyone is expected to follow the `CODE_OF_CONDUCT.md`: be respectful, give kind and specific feedback, assume good intent, and keep things safe for school students. Concerns can be sent to contact@yantrika.in. Security problems should be reported privately (see `SECURITY.md`), not as public issues.

13. Troubleshooting

Problem	Likely cause and fix
My game does not show up	Is its id listed in `games/manifest.js`? Does the folder name match the `id`? Open the browser console (F12) for red errors. Press Ctrl+F5.
Nothing changes when I edit	Browser cache. Press Ctrl+F5. If you opened the file directly, make sure you edited the folder you are viewing.
check-games says "folder is not listed in games/manifest.js"	Add the folder name to the list in `games/manifest.js`.
check-games says "event ... does not exist"	Use only ids from the EV list in `index.html`, or leave `events` empty.
check-games says "not allowed: network request"	Remove `fetch`/XHR/WebSocket. Games must work offline; include data inside your file.
The thumbnail does not appear	Check the `thumb` path and file name (case matters on GitHub Pages), and that the file is in your folder.
"X is not defined" error	A typo, or something your game uses is not available. Only the tools listed in section 6 are shared. Define your own helpers inside your wrapper.
Timers keep running after I close the game	Stop them when `!el.isConnected`, or clear them when a round ends.
My game looks wrong in dark mode	You used fixed colours. Switch to the CSS variables in section 6.
I am stuck	Open an issue or a draft pull request and ask. Beginners' questions are welcome.

14. Cheat sheet

Commands

```
node tools/new-game.js <id> "<Title>" <pillar>   create a game from the template
node tools/check-games.js                        validate (no install needed)
npm install && npm test                          validate + lint + headless test
python3 -m http.server 8000                      serve the site locally

git checkout -b game/<id>                        start a branch
git add games && git commit -m "Add <Title>"    save your work
git push origin game/<id>                        upload it, then open a pull request
```

Minimum viable game

```
(function(){
'use strict';
function play(el){ el.innerHTML='<h2 class="m">Hello</h2>'; }
YK.register({
  id:'hello', pillar:'learn', icon:'\ud83d\udc4b', title:'Hello Bot',
  description:'One short line for the card.',
  events:[], thumb:'games/hello/thumb.png', run:play
});
})();
```

Before you submit

- Folder `games/<id>/` with `game.js` and a thumbnail
- One line added to `games/manifest.js`
- `node tools/check-games.js` reports 0 errors
- Played on a narrow window and in dark mode
- No network calls, cookies or external files
- Credits added for any third-party material
- Pull request template filled in

Thank you for building with us. Every game, fix and translation helps the next student learn something new.